

局域网访问功能 实现技术文档

深圳市出租车出行数据分析系统

文档版本: 1.0 | 日期: 2026-06-28

技术栈: Python Flask + Windows 防火墙 + 网络诊断

本文档详细阐述了从一个仅限本机 localhost 访问的 Flask 项目
改造为同一局域网内任意设备均可访问的完整技术方案
涵盖网络原理、代码实现、防火墙配置、虚拟网卡过滤及 AP 隔离诊断

一 项目背景与目标

1.1 原始状态

本项目是一个基于 Python Flask 的 Web 数据可视化系统，包含分析看板（/）和实时运行态势大屏（/realtime）两个前端页面。开发调试阶段，开发者通过浏览器访问 `http://localhost:5000` 查看效果，所有访问仅限本机。

1.2 需求场景

- 团队成员使用各自的笔记本电脑，无需安装 Python 环境即可查看项目
- 手机/平板等移动设备快速预览大屏效果
- 向客户演示时，大屏可独立运行在一台电脑上，其他人用各自设备观看
- 不依赖公网服务器或内网穿透工具（如 ngrok），纯局域网直连

1.3 最终效果

只需在主机的 Windows 防火墙上放行 Flask 端口，同一局域网（包括手机热点）内的任意设备即可通过浏览器访问主机的局域网 IP 地址（如 `http://10.220.79.99:5000`）打开完整 Web 项目。

二 网络原理深度解析

2.1 IP 地址与网卡绑定

一台计算机可能同时拥有多个 IP 地址，每个 IP 绑定在不同的网络接口（网卡）上。Flask 的监听地址决定了哪些 IP 上的请求会被接收。

127.0.0.1 (localhost) — 仅本机回环

127.0.0.1 是操作系统内置的虚拟回环网卡地址。当 Flask 绑定到 127.0.0.1 时，只有本机发出的请求能到达 Flask，局域网其他设备发送到主机真实 IP 的数据包会被操作系统直接丢弃。

0.0.0.0 (INADDR_ANY) — 所有网卡

0.0.0.0 是一个特殊的通配地址，表示“监听本机所有网络接口”。当 Flask 绑定到 0.0.0.0 时，操作系统会将发往 127.0.0.1、局域网真实 IP 以及其他任何网卡 IP 的 5000 端口请求全部转发给 Flask。这是实现局域网访问的基础前提。

2.2 数据包在局域网中的旅程

以下是一次完整的局域网访问流程，以另一台电脑访问 `http://10.220.79.99:5000/` 为例：

- 步骤 1 — DNS 解析：浏览器从 URL 中提取 IP 地址 10.220.79.99（无需 DNS）
- 步骤 2 — ARP 广播：操作系统发送 ARP 请求“谁是 10.220.79.99？”到局域网
- 步骤 3 — MAC 地址获取：宿主机网卡回复“是我，MAC 地址是 XX:XX:XX:XX”
- 步骤 4 — TCP 三次握手：客户端向宿主机 MAC 地址的 5000 端口发起 SYN
- 步骤 5 — 防火墙检查：Windows 防火墙检查入站规则，若有 TCP 5000 Allow 则放行
- 步骤 6 — Flask 接收：内核将数据包交给监听 0.0.0.0:5000 的 Flask 进程
- 步骤 7 — HTTP 响应：Flask 处理请求 → 返回 HTML → 原路返回给客户端

[i] 整个过程中，Flask 本身无需做任何额外配置——只要 `host="0.0.0.0"` 且防火墙放行，操作系统内核会自动处理所有的

2.3 手机热点的特殊网络拓扑

使用手机热点组网时，手机充当了“路由器 + DHCP 服务器”的角色。连接到同一热点的设备会被分配到同一子网的 IP 地址（如 10.220.79.0/24）。多数手机热点默认会分配 C 类私网地址（192.168.x.x），部分安卓手机使用 A 类私网（10.x.x.x）。

小米手机（如 Xiaomi 14）的热点通常分配 10.x.x.x 网段的地址，这是 RFC 1918 定义的合法私有地址段，与 192.168.x.x 在功能上完全等价。

三 代码实现详解

3.1 Flask 配置 — 监听所有接口

src/config.py 中的关键配置项：

```
FLASK_HOST = "0.0.0.0"    # 监听本机所有网络接口
FLASK_PORT = 5000         # HTTP 服务端口
FLASK_DEBUG = True        # 开发模式（生产环境应设为 False）
```

这是实现局域网访问的唯一前提条件之一。但仅有这个配置还不够——如果 Windows 防火墙拦截了 5000 端口的入站请求，外部设备依然无法连接。

3.2 IP 自动检测 — 排除虚拟网卡

本机可能同时存在多个 IP 地址（WiFi 网卡、VMware 虚拟网卡、VPN 虚拟网卡、Radmin 虚拟网卡等）。如果错误地将 VMware 的 192.168.116.1 显示为局域网地址，其他设备永远无法通过该 IP 访问到本机——因为该 IP 仅存在于虚拟网络中。

方案对比

方案	准确性	复杂度	跨平台	说明
socket.gethostname()	X 差	低	V	无法区分虚拟/真实网卡
ipconfig 文本解析	~ 一般	中	X	GBK 编码不稳定
PowerShell Get-NetIPAddress	V 好	中	X	结构化数据 + 适配器名称
UDP connect 探测	V 好	低	V	获取实际出口 IP

最终方案：PowerShell + UDP 组合

```
def _get_lan_ip():
    """获取本机局域网 IPv4 地址（自动排除虚拟网卡）"""

    # 虚拟网卡关键词黑名单
    VIRTUAL_KEYWORDS = [
        "vmware", "virtualbox", "hyper-v", "vethernet",
        "wsl", "virtual", "tap-", "radmin", "vpn",
        "loopback", "bluetooth", "teredo", "tunnel",
    ]

    # Windows: PowerShell 获取适配器名称 + IP
    ps_cmd = (
        "Get-NetIPAddress -AddressFamily IPv4 | "
        "Select-Object IPAddress, InterfaceAlias | "
        "ConvertTo-Csv -NoTypeInfo"
    )

    result = subprocess.run(
        ["powershell", "-NoProfile", "-Command", ps_cmd],
        capture_output=True, text=True, encoding="utf-8"
    )

    for line in result.stdout.split("\n"):
        ip, alias = parse_csv_line(line)
        # 检查适配器名称是否包含虚拟关键词
        is_virtual = any(kw in alias.lower()
                        for kw in VIRTUAL_KEYWORDS)
        if not is_virtual:
            ips.append(ip)

    # 备选: UDP connect 获取路由出口 IP
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    s.connect(("8.8.8.8", 53)) # Google DNS
    real_ip = s.getsockname()[0]
    s.close()
    return ips
```

3.3 启动信息展示

在 Flask 启动时，自动调用 `_get_lan_ip()` 检测本机真实 IP 并打印访问地址，让用户无需手动查找 IP。启动后的控制台输出效果：

```
=====
城市出行数据分析系统 - Flask Web 服务
=====
```

[Access]

```
└─ 本机访问: http://localhost:5000
└─ 分析看板: http://localhost:5000/
└─ 实时大屏: http://localhost:5000/realtime
```

[LAN] 局域网访问（同局域网设备）：

```
└─ http://10.220.79.99:5000/          （分析看板）
└─ http://10.220.79.99:5000/realtime （实时大屏）
```

四 操作系统防火墙配置

4.1 为什么防火墙是最大障碍

Windows Defender 防火墙默认阻止所有入站连接（除非匹配已有允许规则）。Flask 绑定 0.0.0.0 只解决了"服务端监听"的问题，防火墙则决定了"客户端能否到达服务端"。这是 99% 的局域网访问失败的根本原因。

Windows 防火墙规则要素

参数	值	说明
名称	Taxi Flask Web (5000)	便于识别和管理
方向	Inbound (入站)	外部设备发往本机的请求
协议	TCP	HTTP 基于 TCP 协议
本地端口	5000	Flask 监听端口
操作	Allow (允许)	放行匹配的流量
配置文件	Private + Public	家庭/工作网络 和 公用网络

4.2 自动化配置脚本

本项目的 setup_firewall.bat 自动完成防火墙配置（需管理员权限）：

```
@echo off

:: 检查管理员权限
net session >nul 2>&1
if %errorlevel% neq 0 (
    echo [WARN] 需要管理员权限!
    pause && exit /b 1
)

:: 删除旧规则（如果存在）
netsh advfirewall firewall delete rule \
    name="Taxi Flask Web (5000)" 2>nul

:: 创建新规则
netsh advfirewall firewall add rule \
    name="Taxi Flask Web (5000)" \
    description="Shenzhen Taxi Analysis" \
    dir=in protocol=TCP localport=5000 \
    action=allow profile=private,public

echo [OK] 防火墙规则添加成功!
```

[i] 如果 Windows 防火墙规则正确但依然不通，还需检查第三方安全软件（如 360、火绒、McAfee）是否拦截了入站连接。

4.3 Linux 防火墙（CentOS 7）

```
#!/bin/bash
# 自适应检测防火墙类型
if firewall-cmd --state 2>/dev/null; then
    sudo firewall-cmd --permanent --add-port=5000/tcp
    sudo firewall-cmd --reload
elif ufw status 2>/dev/null; then
    sudo ufw allow 5000/tcp
else
    sudo iptables -I INPUT -p tcp --dport 5000 -j ACCEPT
fi
```

五 诊断工具与排错指南

5.1 一键诊断脚本

本项目的 `diagnose_network.bat` 提供了 4 步自动化诊断，覆盖了局域网访问失败的全部可能原因：

步骤	检查项	工具/命令	可能的失败原因
Step 1	IP 地址正确性	PowerShell Get-NetIPAddress	虚拟网卡 IP 混淆 → 排除 VMware/VPN
Step 2	端口监听状态	<code>netstat -ano findstr :5000</code>	Flask 未启动或绑定 127.0.0.1
Step 3	防火墙规则	<code>netsh advfirewall show rule</code>	规则不存在或被禁用
Step 4	AP 隔离检查	提示用户检查手机设置	手机热点阻止设备间通信

5.2 排错清单 (Checklist)

- ① Flask 绑定地址
 - 确认 `src/config.py` 中 `FLASK_HOST = "0.0.0.0"` (不是 "127.0.0.1")
- ② 防火墙规则
 - 以管理员身份运行 `setup_firewall.bat`
- ③ 端口监听
 - `netstat -ano | findstr ":5000 LISTENING"` 应有输出
- ④ IP 正确性
 - `diagnose_network.bat` 输出的 [真实] IP 是否为你网络的实际 IP
- ⑤ 同网段
 - 其他设备和宿主机必须在同一子网 (检查 IP 前缀是否相同)
- ⑥ AP 隔离
 - 手机热点设置中确认 "允许设备互联" 已开启或 "AP 隔离" 已关闭
- ⑦ 三方防火墙
 - 暂时禁用 360/火绒/卡巴斯基 等安全软件测试
- ⑧ Ping 测试
 - 从另一设备 ping 宿主机 IP, 确认基本网络连通性
- ⑨ 浏览器访问
 - 在另一设备浏览器输入 `http://宿主机IP:5000/api/health`

5.3 AP 隔离：最隐蔽的障碍

[!] AP 隔离（客户端隔离）是手机热点最常见的"局域网访问拦截器"！

AP 隔离（Access Point Isolation，也叫 Client Isolation）是 Wi-Fi 热点的一项安全功能。开启后，连接到同一热点的设备之间被逻辑隔离，每个设备只能与热点（即手机）通信，无法与其他设备直接交互。这原本是为了公共 Wi-Fi 安全设计的，但会阻碍局域网互访。

小米/Redmi 手机检查路径：

- ＞ 设置 → 个人热点 → 查看是否有"允许设备互联"开关
- ＞ 或 设置 → 连接与共享 → 个人热点 → 更多设置
- ＞ 如果找不到该选项，尝试更换热点名称后重新连接

其他品牌手机：

- ＞ iPhone：个人热点默认无 AP 隔离，通常不需要额外设置
- ＞ 华为/荣耀：设置 → 移动网络 → 个人热点 → 更多 → 允许设备互联
- ＞ OPPO/vivo：设置 → 连接与共享 → 个人热点 → AP 频段/管理

六 相关文件与资源

以下是与局域网访问功能相关的所有文件及其作用：

文件	类型	作用
src/config.py	配置	FLASK_HOST="0.0.0.0" — 监听所有网络接口
src/web_app.py	核心代码	_get_lan_ip() — 智能 IP 检测 + 启动提示
main.py	核心代码	_get_lan_ips() — 命令行模式 LAN IP 显示
setup_firewall.bat	工具脚本	Windows 防火墙一键放行 5000 端口
setup_firewall.sh	工具脚本	Linux 防火墙适配 (firewalld/ufw/iptables)
diagnose_network.bat	诊断工具	4 步自动化网络诊断 + 自动修复

6.1 项目完整启动流程

```
# ===== 第一步：配置防火墙（首次，需管理员） =====
右键 setup_firewall.bat → 以管理员身份运行

# ===== 第二步：启动 Flask 项目 =====
. venv_py38\Scripts\activate
python main.py --web

# ===== 第三步：查看控制台输出的 LAN 地址 =====
# 会显示类似：
# [LAN] 局域网访问（同局域网设备）：
#   └─ http://10.220.79.99:5000/
#   └─ http://10.220.79.99:5000/realtime

# ===== 第四步：其他设备浏览器访问 =====
# 手机/平板/其他电脑打开浏览器
# 输入 http://10.220.79.99:5000/
```

6.2 如果还是不通

```
# 运行诊断脚本（管理员权限）
右键 diagnose_network.bat → 以管理员身份运行

# 根据输出的 [OK]/[FAIL] 状态定位问题
# 最常见的三个原因：
# 1. 防火墙未放行 → 脚本自动修复
# 2. AP 隔离 → 手动修改手机设置
# 3. IP 错误（虚拟网卡） → 脚本自动显示正确 IP
```

附录 网络基础概念速查

RFC 1918 私有地址段

以下三个地址段被 IANA 保留用于私有网络，在公网不可路由：

地址范围	CIDR	子网掩码	可用主机数	常见场景
10.0.0.0 - 10.255.255.255	10.0.0.0/8	255.0.0.0	16,777,214	企业内网/手机热点
172.16.0.0 - 172.31.255.255	172.16.0.0/12	255.240.0.0	1,048,574	中型企业
192.168.0.0 - 192.168.255.255	192.168.0.0/16	255.255.0.0	65,534	家庭路由器

OSI 七层模型视角下的局域网访问

层级	名称	本项目的对应操作
L7	应用层	HTTP 请求 → Flask 路由 → Jinja2 渲染 HTML
L6	表示层	UTF-8 文本 → JSON 序列化/反序列化
L5	会话层	TCP 连接管理（三次握手/四次挥手）
L4	传输层	TCP 端口 5000 → 进程 PID 绑定
L3	网络层	IP 路由（0.0.0.0 → 局域网 IP 转发）
L2	数据链路层	ARP 解析 MAC 地址 → 以太网帧传输
L1	物理层	Wi-Fi 无线电波 / 以太网双绞线

关键命令速查表

命令	用途	示例输出
<code>netstat -ano findstr :5000</code>	查看端口监听	TCP 0.0.0.0:5000 LISTENING
<code>netsh advfirewall firewall show rule</code>	查看防火墙规则	规则名称/端口/操作
<code>ipconfig</code>	查看所有IP	IPv4 地址列表 + 适配器名
<code>ping <对方IP></code>	测试连通性	Reply from ... time<1ms
<code>curl http://IP:5000/api/health</code>	测试HTTP	<code>{"status": "ok"}</code>